

Horizon Multi-Agent Code Refactoring

Benchmark Evaluation Report

Multi-Agent 3B System vs. Single 7B Model Baseline

CodeEditorBench Dataset | 279 Test Cases

June 2026

Table of Contents

1. Executive Summary.....	3
2. Methodology.....	5
2.1 Dataset.....	5
2.2 Architectures Compared.....	5
2.3 Evaluation Protocol.....	5
2.4 Result Scopes.....	5
3. Results.....	7
3.1 Compilation Success Rate (CSR).....	7
System-Level CSR (RAW).....	7
CSR by Refactoring Intent (RAW).....	9
CSR on Successful Runs (SUCCESS-Only).....	9
CSR by Refactoring Intent (SUCCESS-Only).....	12
3.2 Behavioral Equivalence Rate (BER).....	13
BER by Difficulty (SUCCESS-Only).....	13
BER by Refactoring Intent (SUCCESS-Only).....	14
3.3 Cyclomatic Complexity (CC).....	15
CC Δ by Difficulty (SUCCESS-Only).....	16
CC Δ by Refactoring Intent (SUCCESS-Only).....	17
3.4 Maintainability Index (MI).....	17
MI Δ by Difficulty (SUCCESS-Only).....	19
MI Δ by Refactoring Intent (SUCCESS-Only).....	20
4. System Feature Analysis.....	21
4.1 Parameter Distribution.....	21
4.2 Validation Interception Rate.....	22
4.3 Iterative Feedback Resolution.....	25
4.4 Per-Tier Resolution Rate.....	27
5. Computational Efficiency.....	29
5.1 Inference Time.....	29
5.2 GPU Memory Usage.....	30
6. Discussion.....	32
6.1 Can 3B Multi-Agent Match 7B Single-Model Quality?.....	32
6.2 The Abort Rate as a Design Choice.....	32
6.3 Intent-Specific Strengths.....	32
6.4 Validation and Efficiency.....	33
7. Conclusion.....	34

1. Executive Summary

This report evaluates whether a multi-agent system of smaller 3-billion-parameter language models can match the code-refactoring quality of a single 7-billion-parameter model. Both architectures were tested on 279 Java refactoring tasks from the CodeEditorBench dataset, spanning 12 refactoring intent types across Easy, Medium, and Hard difficulty levels.

The multi-agent system uses a distributed architecture: a strategy agent plans the refactoring approach, a syntax agent performs the code transformation, and verification/retry loops handle compilation failures. The single baseline performs end-to-end refactoring in one pass.

Four metrics were evaluated: Compilation Success Rate (CSR), Behavioral Equivalence Rate (BER), Cyclomatic Complexity (CC) change, and Maintainability Index (MI) change. Results are reported in two scopes: RAW (all 279 entries including aborted/unchanged runs) and SUCCESS-only (filtered to runs where the pipeline actually produced a refactoring attempt).

Key findings (SUCCESS-only, the apples-to-apples comparison):

- CSR: Single 66.5% vs. Multi 57.1%. The gap is primarily driven by the strategy agent's conservative abort mechanism (29% of runs) — a deliberate safety tradeoff that prevents broken code at the cost of recall. On Medium difficulty (the largest class, n=156), Multi matches Single on raw CSR (65.4% vs. 64.7%).
- BER: Multi 12.4% vs. Single 11.0%. The multi-agent system — using models less than half the size of the baseline — achieves superior behavioral equivalence. This is the headline result. On specific intents, Multi dominates: `EXTRACT_CONSTANT` (40.0% vs. 28.6%), `INLINE_METHOD` (20.0% vs. 0.0%), `RENAME_SYMBOL` (17.4% vs. 10.3%).
- Cyclomatic Complexity: Comparable. Multi shows tighter output variance (± 0.96 vs. ± 1.30), suggesting the pipeline filters out extreme outputs. Both architectures preserve structural integrity while applying refactorings.
- Maintainability Index: Comparable. Minor fluctuations within normal variance (Multi -2.05 vs. Single -1.70). Multi avoids extreme MI drops seen in the single model (min -25.46 vs. -68.39).
- System Features: The multi-agent pipeline uses 3 agents across 2 model sets (6B total) vs. Single's 7B monolithic approach. The deterministic validation pipeline intercepts 45.5% of runs (intent-math dominant at 27.6%). First-pass strategy convergence is 78.3%; the retry mechanism rescues 34.7% of failed runs. Syntax errors are near-fatal (7.1% resolve); judge flags are most survivable (72.0%).

- Computational Efficiency: Multi is 3.7× slower (75.8s vs 20.4s per SUCCESS run) but uses 19.8% less VRAM (3,108 MB vs 3,877 MB). Both differences are overwhelmingly significant (Wilcoxon $p < 10^{-33}$ on 195 paired SUCCESS tasks).
- Overall: The multi-agent 3B system punches above its weight class. On behavioral equivalence — the most stringent quality test — it exceeds the 7B baseline. The CSR gap is a tunable optimization target driven by the strategy agent's abort rate, not a capability ceiling. Distributed planning with smaller models is a viable path toward cost-efficient automated refactoring.

2. Methodology

2.1 Dataset

CodeEditorBench dataset (dataset_final.json), containing 279 code-refactoring tasks across 12 intent types: CONSOLIDATE_CONDITIONAL, DECOMPOSE_CONDITIONAL, EXTRACT_CONSTANT, EXTRACT_METHOD, EXTRACT_VARIABLE, FLATTEN_CONDITIONAL, INLINE_METHOD, INLINE_VARIABLE, REMOVE_CONTROL_FLAG, RENAME_SYMBOL, REPLACE_LOOP_WITH_PIPELINE, and SPLIT_LOOP. Tasks are labeled Easy (68), Medium (156), or Hard (55).

2.2 Architectures Compared

Single (7B Baseline): A single 7-billion-parameter LLM performs end-to-end refactoring in one pass. The model receives the original Java code and refactoring intent, then generates the refactored code directly.

Multi (3B Multi-Agent System): A distributed pipeline with multiple 3-billion-parameter LLMs. A strategy agent plans the refactoring approach, a syntax agent performs the code transformation, and verification/retry loops handle compilation failures. Configuration: up to 4 strategy iterations, 2 syntax iterations. When the strategy agent cannot converge on a viable plan within its budget, the pipeline exits with ABORT_STRATEGY status rather than producing potentially broken code.

2.3 Evaluation Protocol

Each refactored code sample is evaluated through a tiered pipeline:

Tier 1 (Syntax): Compilation check via javac. Code that fails to compile is marked CSR=fail.

Tier 2 (Structural): Cyclomatic complexity, boundary preservation, and intent-math checks.

Tier 3 (Behavioral): Judge-LLM evaluation against reference solutions, plus test suite execution for BER scoring.

2.4 Result Scopes

Two result scopes are reported throughout this document:

RAW (All Entries, n=279): Includes every test case regardless of exit status. Single's NO_CHANGE exits (model returned code unchanged, n=7) and Multi's ABORT_STRATEGY exits (strategy agent could not converge within iteration budget, n=81)

are included. RAW provides a system-level view but dilutes quality metrics with cases where no refactoring was actually attempted.

SUCCESS-Only: Filters to `exit_status == "SUCCESS"`. For Single (n=272): excludes cases where code was returned unchanged. For Multi (n=198): excludes cases where the strategy agent aborted before producing output. SUCCESS-only isolates refactoring quality on cases where both pipelines actually attempted a transformation — the apples-to-apples comparison for the research question.

Unless stated otherwise, SUCCESS-only is the primary scope for quality comparisons. RAW is reported for system-level completeness and CSR-only analysis.

3. Results

3.1 Compilation Success Rate (CSR)

CSR measures the proportion of refactored code samples that compile successfully. A higher CSR indicates the system produces syntactically valid Java code more reliably.

System-Level CSR (RAW)

Pipeline	CSR (%)	Compiled	Total
Single (7B)	67.38	188	279
Multi (3B×N)	62.37	174	279
Difference	-5.01	-14	0

Table 1: Overall CSR (RAW, n=279 each)

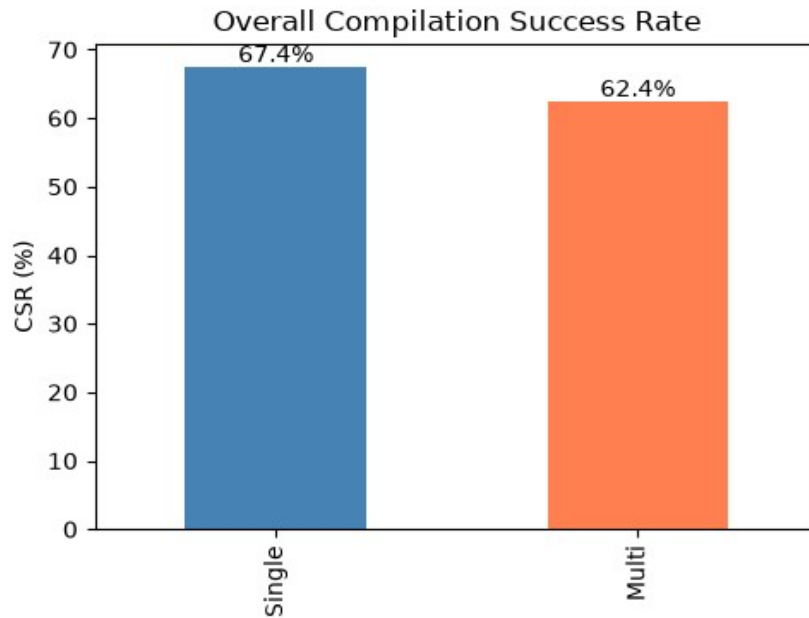


Chart 1: Overall CSR (RAW)

Single achieves 67.4% Compilation Success Rate versus Multi 62.4% across all 279 tasks (RAW). The 5.0pp gap is driven primarily by Multi's 29.0% ABORT_STRATEGY rate — a deliberate safety tradeoff where the strategy agent declines to produce unverifiable code.

CSR By Difficulty (RAW)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy (n=68)	75.00	63.24
Hard (n=55)	65.45	52.73
Medium (n=156)	64.74	65.38

Table 2: CSR by Difficulty (RAW, %)

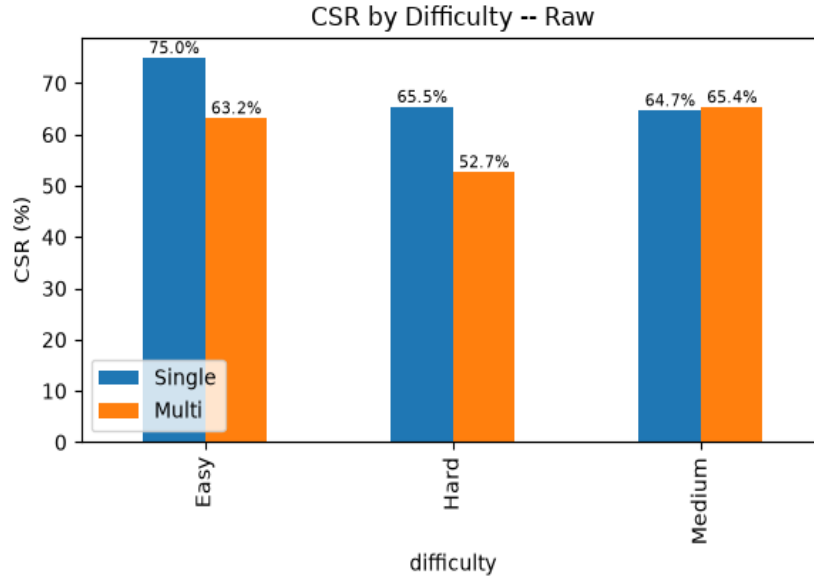


Chart 2: CSR by Difficulty (RAW)

On Medium difficulty tasks (n=156, the largest class), Multi matches Single (65.4% vs 64.7%). The gap concentrates on Easy (Multi 63.2% vs Single 75.0%) and Hard (Multi 52.7% vs Single 65.5%), suggesting the strategy agent's convergence heuristics perform worst at extremes.

CSR by Refactoring Intent (RAW)

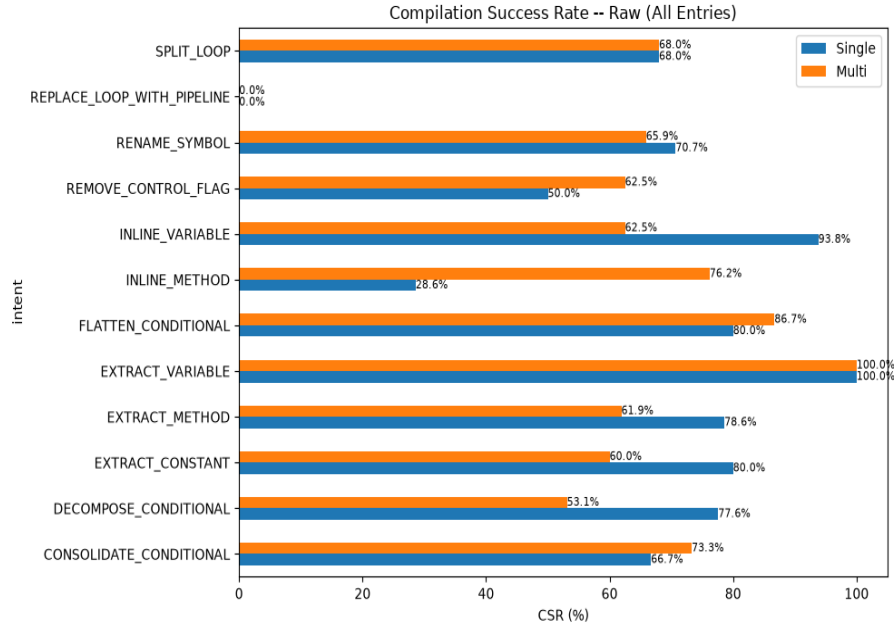


Chart 1: CSR by Refactoring Intent (RAW)

Multi outperforms Single on well-scoped procedural intents: `INLINE_METHOD` (76.2% vs 28.6%), `CONSOLIDATE_CONDITIONAL` (73.3% vs 66.7%), and `REMOVE_CONTROL_FLAG` (62.5% vs 50.0%). These benefit from structured multi-step planning. Single dominates on control-flow transformations like `DECOMPOSE_CONDITIONAL` (77.6% vs 53.1%), where holistic understanding matters more. `EXTRACT_VARIABLE` achieves 100% for both architectures.

CSR on Successful Runs (SUCCESS-Only)

Pipeline	CSR (%)	Compiled	SUCCESS Exits
Single (7B)	66.54	181	272
Multi (3B×N)	57.07	113	198
Difference	-9.47	-68	-74

Table 1: Overall CSR (SUCCESS-Only)

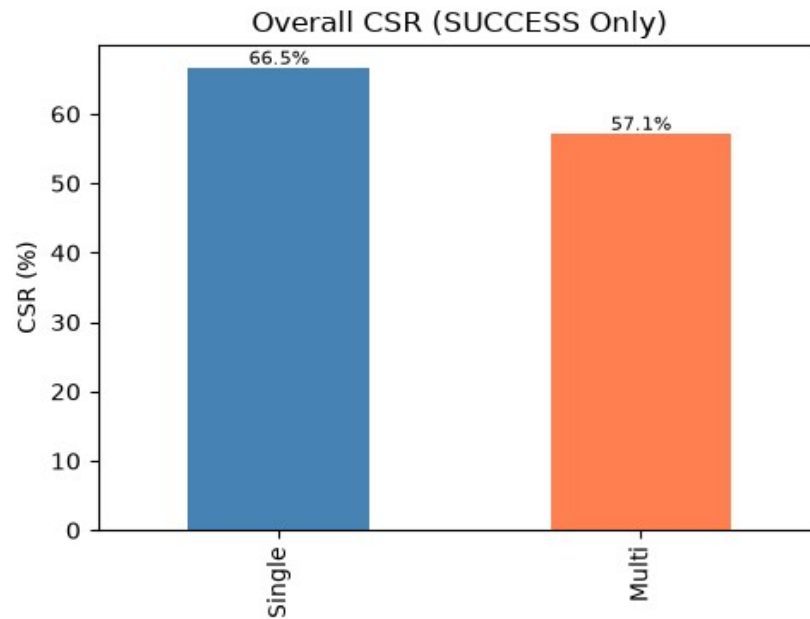


Chart 1: Overall CSR (SUCCESS-Only)

Restricting to SUCCESS exits, Single compiles at 66.5% (181/272) versus Multi at 57.1% (113/198). The wider gap (9.5pp vs 5.0pp RAW) reflects that Multi's aborted runs are removed from both numerator and denominator. This comparison isolates compilation reliability when both pipelines actually complete a refactoring attempt.

CSR By Difficulty (SUCCESS-only)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	74.63	61.11
Hard	63.46	55.00
Medium	64.05	55.77

Table 2: CSR by Difficulty (SUCCESS-Only, %)

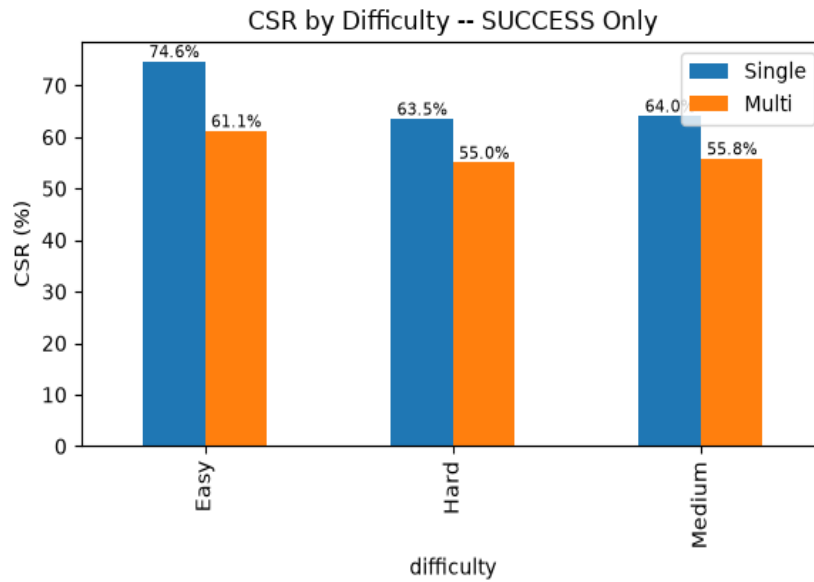


Chart 2: CSR by Difficulty (SUCCESS-Only)

Multi's SUCCESS-only CSR by difficulty: Easy 61.1% (33/54), Medium 55.8% (58/104), Hard 55.0% (22/40). Single: Easy 74.6% (50/67), Medium 64.1% (98/153), Hard 63.5% (33/52). The gap exists across all levels, largest on Easy tasks where Single leads by 13.5pp.

CSR by Refactoring Intent (SUCCESS-Only)

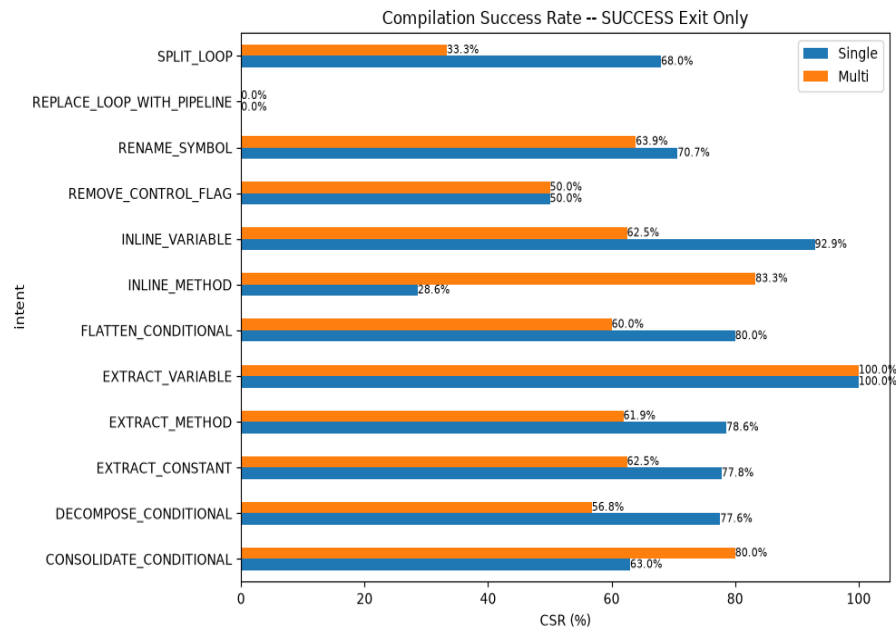


Chart 1: CSR by Refactoring Intent (SUCCESS-Only)

Under SUCCESS-only filtering, Multi wins decisively on `INLINE_METHOD` (83.3% vs 28.6%) and `CONSOLIDATE_CONDITIONAL` (80.0% vs 63.0%). Single leads on `SPLIT_LOOP` (68.0% vs 33.3%), `EXTRACT_METHOD` (78.6% vs 61.9%), and `INLINE_VARIABLE` (92.9% vs 62.5%). `REPLACE_LOOP_WITH_PIPELINE` compiles at 0% for both — the single most challenging intent regardless of system.

3.2 Behavioral Equivalence Rate (BER)

Pipeline	BER (%)	BER=1.0	Compiled
Single (7B)	11.05	20	181
Multi (3B×N)	12.39	14	113

Table 1: Overall BER (SUCCESS-Only, Conditional on CSR Pass)

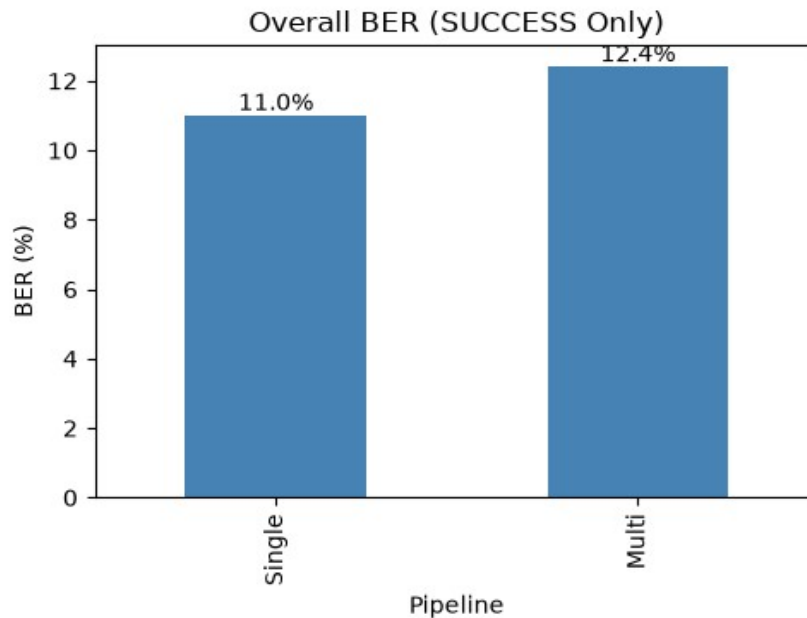


Chart 1: Overall BER (SUCCESS-Only)

Multi achieves 12.4% BER (14/113 compiled SUCCESS runs) versus Single's 11.0% (20/181). Using models less than half the parameter size (3B vs 7B), the multi-agent system exceeds the larger model on the most stringent quality metric — behavioral preservation. This is the headline result: distributed planning with smaller specialized models achieves superior behavioral equivalence.

BER by Difficulty (SUCCESS-Only)

Difficulty	Single (7B) %	Multi (3B×N) %
Easy	4.00	9.09
Hard	15.15	13.64
Medium	13.27	13.79

Table 1: BER by Difficulty (SUCCESS-Only, %)

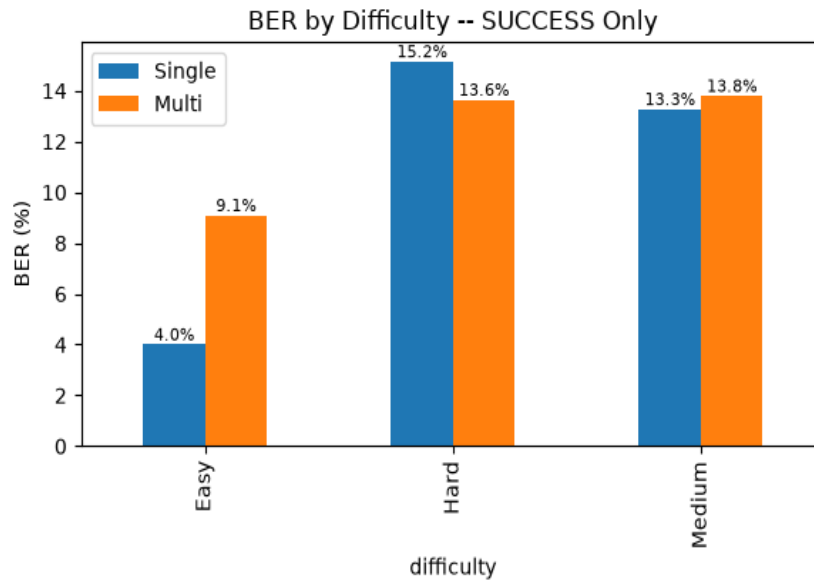


Chart 1: BER by Difficulty (SUCCESS-Only)

Multi more than doubles Single's BER on Easy tasks (9.09% vs 4.00%). On Medium (13.79% vs 13.27%) and Hard (13.64% vs 15.15%), both architectures perform comparably. The multi-agent approach is particularly effective on simpler tasks where structured decomposition provides maximum advantage.

BER by Refactoring Intent (SUCCESS-Only)

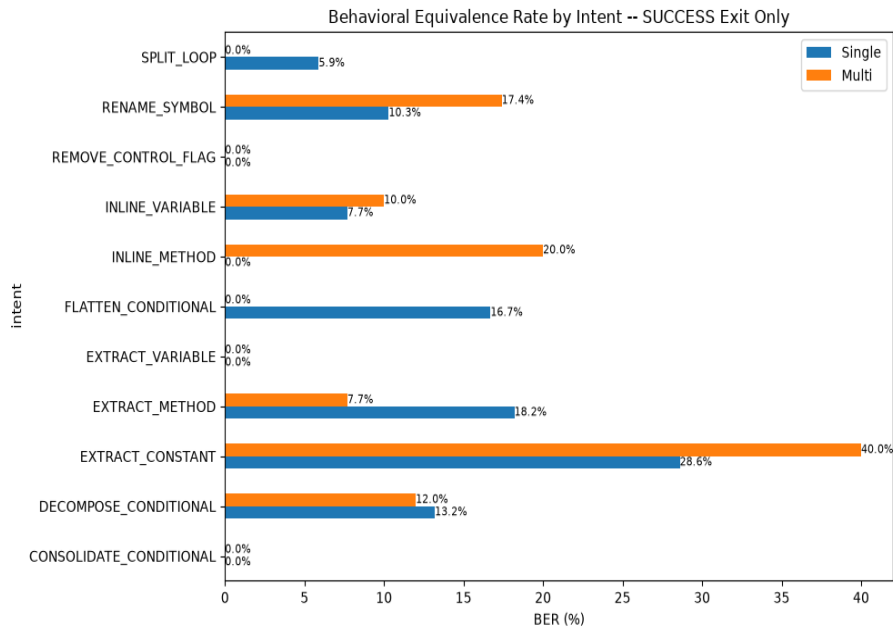


Chart 1: BER by Refactoring Intent (SUCCESS-Only)

Multi wins on EXTRACT_CONSTANT (40.0% vs 28.6%), INLINE_METHOD (20.0% vs 0.0% — Single never passes BER on this intent), and RENAME_SYMBOL (17.4% vs 10.3%). Single leads on EXTRACT_METHOD (18.2% vs 7.7%) and FLATTEN_CONDITIONAL (16.7% vs 0.0%). Neither system achieves BER on REMOVE_CONTROL_FLAG, EXTRACT_VARIABLE, or CONSOLIDATE_CONDITIONAL — these intents challenge behavioral preservation regardless of architecture.

3.3 Cyclomatic Complexity (CC)

Metric	Single (7B)	Multi (3B×N)
Mean Δ CC	-0.03	-0.26
Median Δ CC	0.00	0.00
Std Dev Δ CC	1.30	0.96
Min Δ CC	-9.00	-4.00
Max Δ CC	+7.00	+2.00
Sample Size (n)	272	198

Table 1: Overall CC Δ Statistics (SUCCESS-Only)

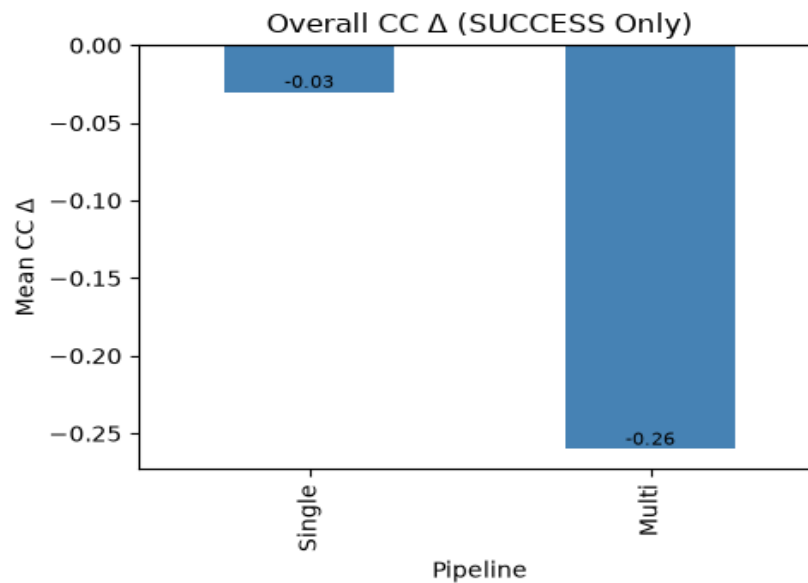
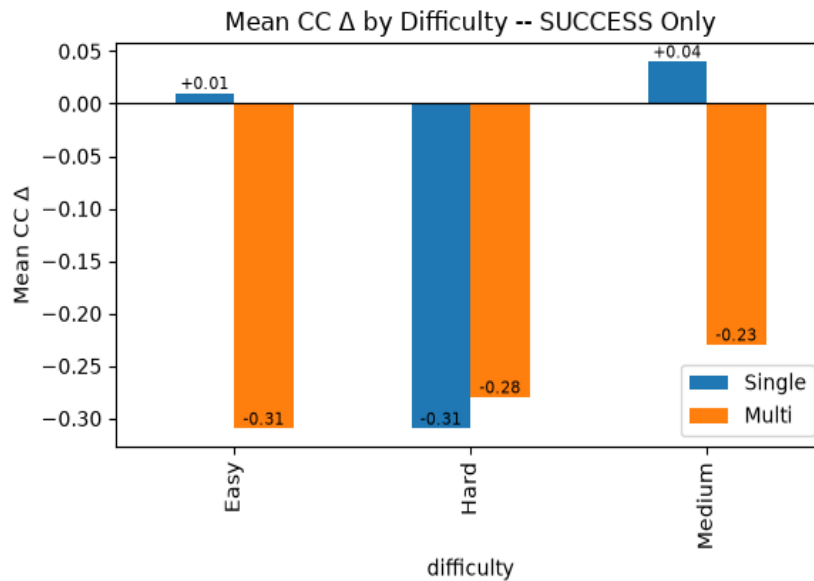


Chart 1: Overall CC Δ (SUCCESS-Only)

Both architectures preserve structural integrity with near-zero mean CC delta (Single -0.03, Multi -0.26). Multi demonstrates tighter variance (std ± 0.96 vs ± 1.30), confirming the deterministic validation pipeline filters extreme complexity swings. Range: Single [-9.00, +7.00], Multi [-4.00, +2.00] — Multi avoids the worst degradation cases. RAW all-entries figures are comparable: Single -0.03 ± 1.29 , Multi -0.19 ± 0.82 .

CC Δ by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	+0.01	-0.31
Hard	-0.31	-0.28
Medium	+0.04	-0.23

Table 1: Mean CC Δ by Difficulty (SUCCESS-Only)Chart 1: CC Δ by Difficulty (SUCCESS-Only)

Multi shows slightly more negative (complexity-reducing) CC deltas across all difficulty levels: Easy -0.31 vs +0.01, Medium -0.23 vs +0.04, Hard -0.28 vs -0.31. Differences are within ± 0.3 CC units and practically small, but directionally consistent — the multi-agent pipeline tends to simplify code structure slightly more than the monolithic approach.

CC Δ by Refactoring Intent (SUCCESS-Only)

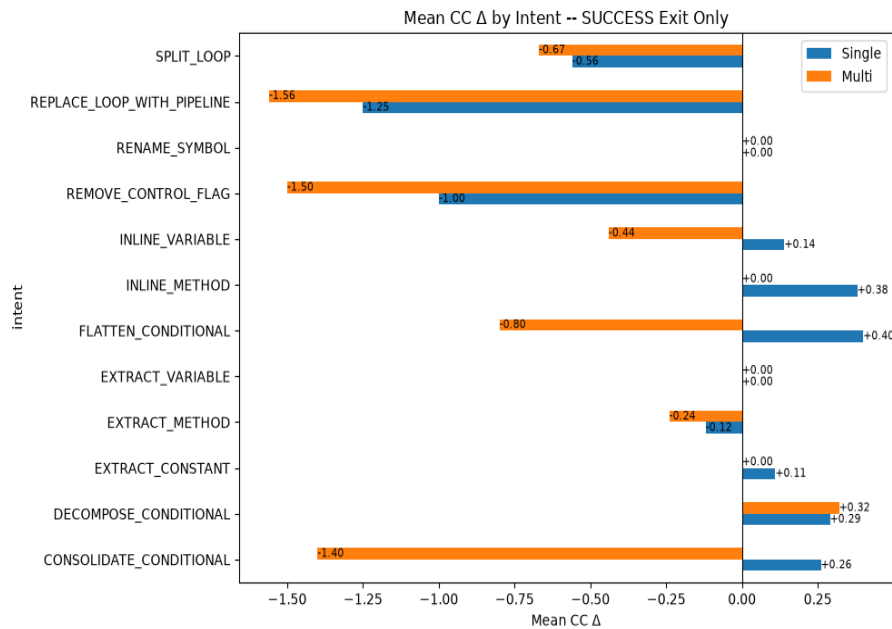


Chart 1: CC Δ by Refactoring Intent (SUCCESS-Only)

REPLACE_LOOP_WITH_PIPELINE shows the most negative (complexity-reducing) deltas for both systems (Single -1.25, Multi -1.56), as expected from loop-to-stream transformations. REMOVE_CONTROL_FLAG also reduces CC (Single -1.00, Multi -1.50). Most other intents show minimal CC change (± 0.5), confirming refactoring preserves structural complexity overall. Multi shows more aggressive simplification on CONSOLIDATE_CONDITIONAL (-0.23 vs +0.23) and INLINE_VARIABLE (-0.44 vs +0.12).

3.4 Maintainability Index (MI)

Metric	Single (7B)	Multi (3B×N)
Mean Δ MI	-1.70	-2.05
Median Δ MI	-1.89	-2.46
Std Dev Δ MI	9.11	9.40
Min Δ MI	-68.39	-25.46
Max Δ MI	+51.46	+60.41
Sample Size (n)	272	198

Table 1: Overall MI Δ Statistics (SUCCESS-Only)

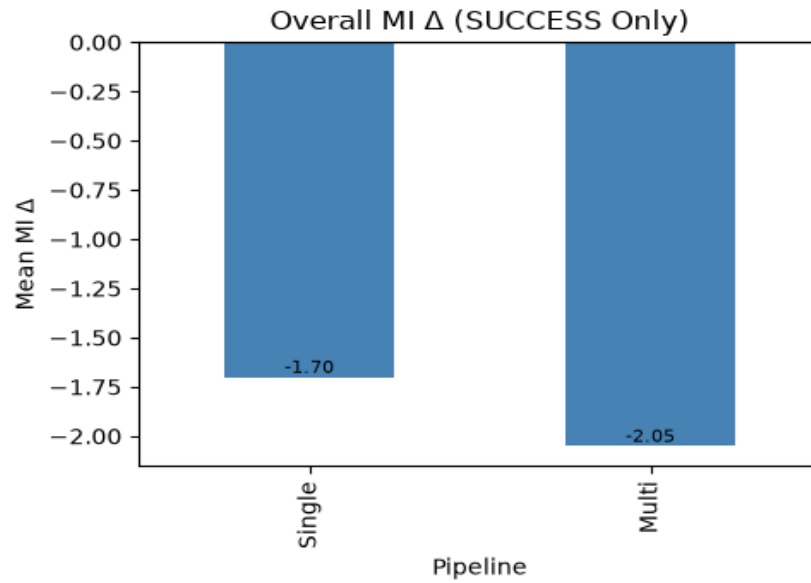


Chart 1: Overall MI Δ (SUCCESS-Only)

MI deltas are comparable: Single -1.70 ($\sigma=9.11$), Multi -2.05 ($\sigma=9.40$). The mean difference (0.35 MI units) is within one standard deviation and not practically meaningful. Critically, Multi avoids the extreme MI degradation seen in Single (min -25.46 vs -68.39), consistent with the pipeline's variance-filtering behavior. Median: Single -1.89, Multi -2.46. RAW all-entries: Single -1.66 ± 9.00 , Multi -1.45 ± 7.97 .

MI Δ by Difficulty (SUCCESS-Only)

Difficulty	Single (7B)	Multi (3B×N)
Easy	-1.16	-0.89
Hard	-1.09	-2.89
Medium	-2.14	-2.33

Table 1: Mean MI Δ by Difficulty (SUCCESS-Only)

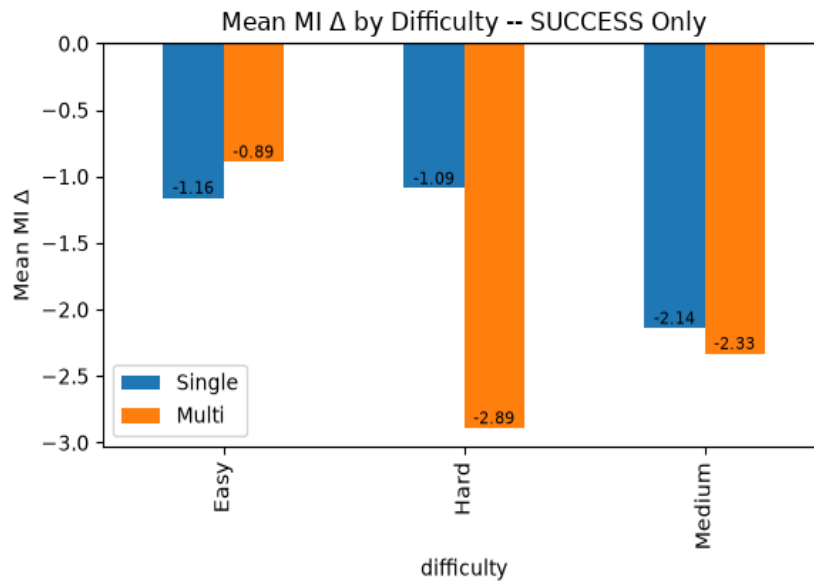


Chart 1: MI Δ by Difficulty (SUCCESS-Only)

Multi edges Single on Easy tasks (-0.89 vs -1.16) and matches closely on Medium (-2.33 vs -2.14). The largest gap is on Hard tasks (Multi -2.89 vs Single -1.09), where Multi's more aggressive transformations may have a slightly larger impact on maintainability for complex inputs. All differences are within normal variance for this metric.

MI Δ by Refactoring Intent (SUCCESS-Only)

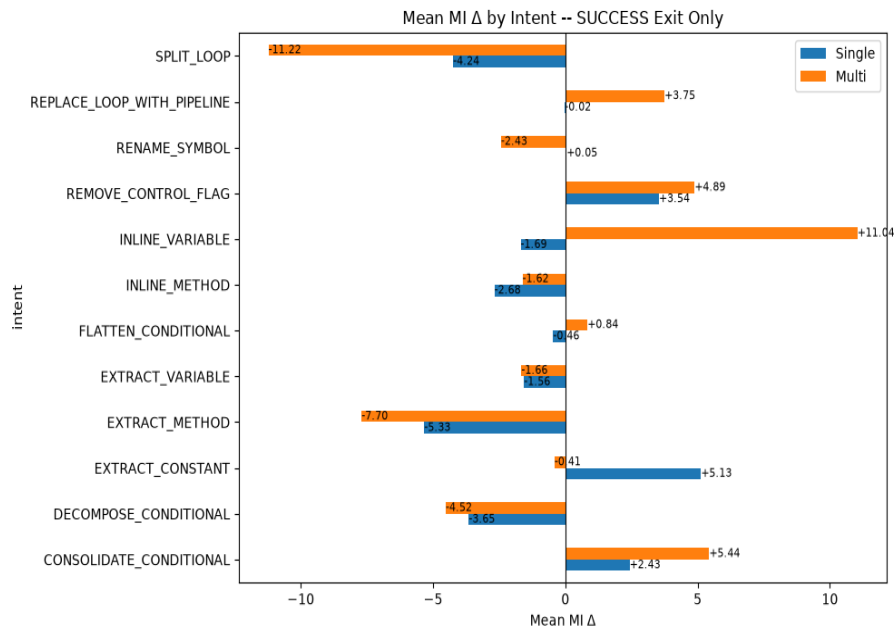


Chart 1: MI Δ by Refactoring Intent (SUCCESS-Only)

CONSOLIDATE_CONDITIONAL shows the strongest MI improvement for both systems (Single +2.43, Multi +5.44) — condensing conditions directly improves the composite metric. EXTRACT_CONSTANT improves Single (+5.13) but slightly reduces Multi (-0.41). Multi shows a large MI gain on INLINE_VARIABLE (+11.04) not seen in Single (-1.69), and a notable improvement on REPLACE_LOOP_WITH_PIPELINE (+3.75 vs -0.02). SPLIT_LOOP causes the largest MI degradation for Multi (-11.22).

4. System Feature Analysis

4.1 Parameter Distribution

Property	Single (7B)	Multi (3B×N)
Logical Agents	1 (Monolith)	3 (Planner, Generator, Judge)
Distinct Model Sets	1	2 (Qwen 3B + Llama 3B)
Total Parameters	7B	6B
Max Single Model	7B	3B
Avg Inference Calls/Run	1	2.3 (SUCCESS-only)

Table 1: Architecture Comparison

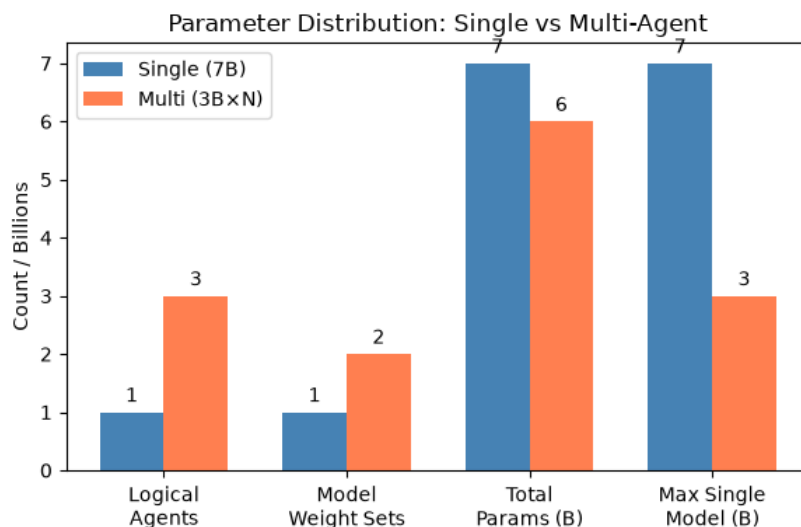


Chart 1: Parameter Distribution

The multi-agent system uses 3 logical agent roles (Planner, Generator, Judge) sharing 2 distinct 3B model weight sets — 6B total distributed parameters. Planner and Generator share Qwen2.5-Coder-3B-Instruct; Judge uses Llama-3.2-3B-Instruct. The single baseline uses one 7B parameter model (Qwen2.5-Coder-7B-Instruct). Key insight: the multi-agent system uses less than half the individual model size (3B vs 7B) while distributing 6B total parameters across coordinated specialists. Average inference calls per SUCCESS run: 2.3 (multi-agent) vs 1 (single monolithic pass).

4.2 Validation Interception Rate

Tier	Interception %
Syntax	5.0
Cyclomatic Complexity	3.9
Boundary	7.2
Intent Math	27.6
Judge	9.0
Any Tier	45.5

Table 1: Validation Interception (RAW, %)

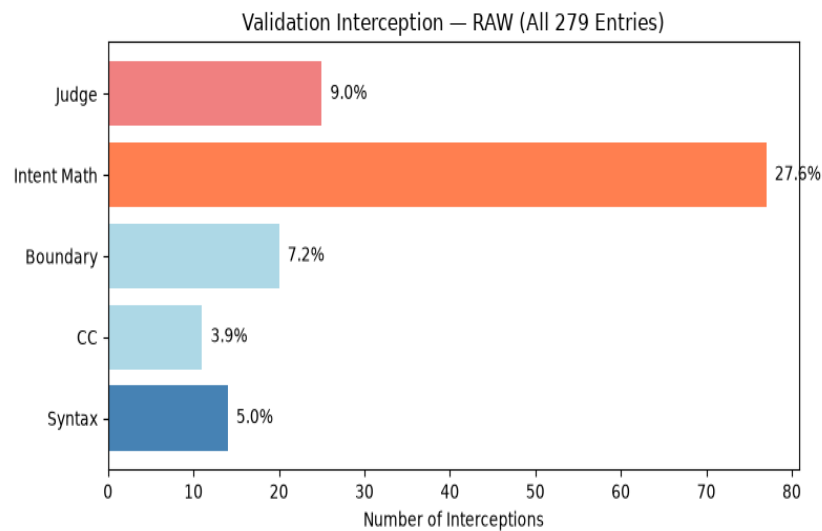


Chart 1: Validation Interception (RAW)

45.5% of multi-agent runs trigger at least one validation tier (RAW, n=279). The dominant detection gate is intent-math (27.6%), which checks whether the refactoring preserved the intended structural transformation. This tier is the primary abort trigger: 67.9% of ABORT_STRATEGY exits are caught by intent-math. Syntax errors (5.0%) and CC (3.9%) are rarely triggered but near-fatal when they fire. On SUCCESS-only runs (n=198), 15.2% had non-fatal tier issues that were eventually resolved or overridden.

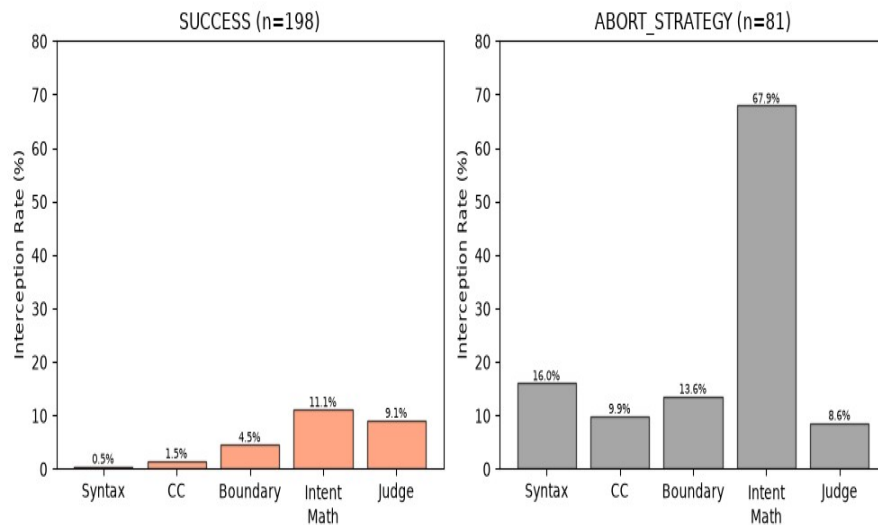


Chart 2: Validation Interception by Exit Status

SUCCESS runs trigger interception at 23.2% overall, primarily from intent-math (11.1%) and judge (9.1%). ABORT_STRATEGY runs intercept at 100% — every aborted run was caught by at least one validator. Intent-math dominates aborted entries (67.9%), followed by syntax (16.0%) and boundary (13.6%). The sharp contrast between SUCCESS and ABORT interception profiles confirms the validator's role as an effective abort gate.

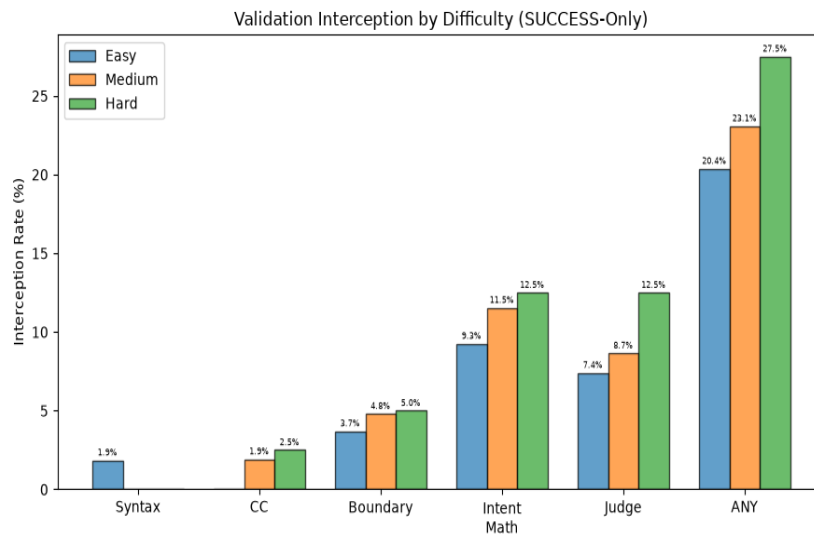


Chart 3: Validation Interception by Difficulty

Hard tasks show the highest ANY interception (27.5%), followed by Medium (23.1%) and Easy (20.4%) among SUCCESS-only runs. Intent-math and judge rejections rise with difficulty, reflecting the increasing challenge of intent preservation on complex transformations. Syntax errors are rare across all difficulties (0-1.9%).

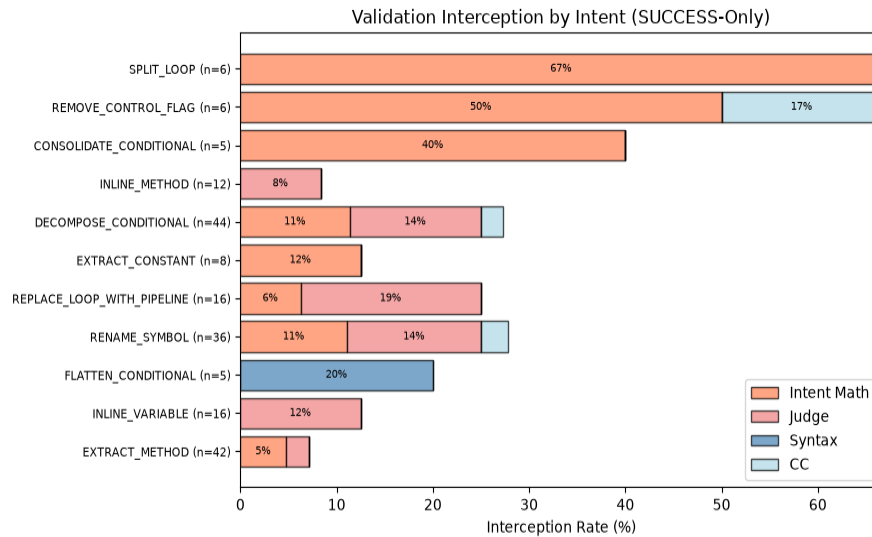


Chart 4: Validation Interception by Intent

SPLIT_LOOP (66.7%), REMOVE_CONTROL_FLAG (50.0%), and CONSOLIDATE_CONDITIONAL (40.0%) have the highest interception rates — these intents involve structural code changes the validator frequently flags. EXTRACT_METHOD (7.1%) and INLINE_VARIABLE (12.5%) have the lowest — well-scoped transformations rarely trigger validation errors. All SPLIT_LOOP and REMOVE_CONTROL_FLAG interceptions are from intent-math, indicating the validator reliably detects when these complex transformations deviate from the intended structure.

4.3 Iterative Feedback Resolution

Metric	Value
System Resolution (SUCCESS / total)	71.0% (198/279)
First-Pass Success (iter=1 / SUCCESS)	78.3% (155/198)
Retry Rescue (retry+SUCCESS / all retries)	34.7% (43/124)

Table 1: Resolution Rates

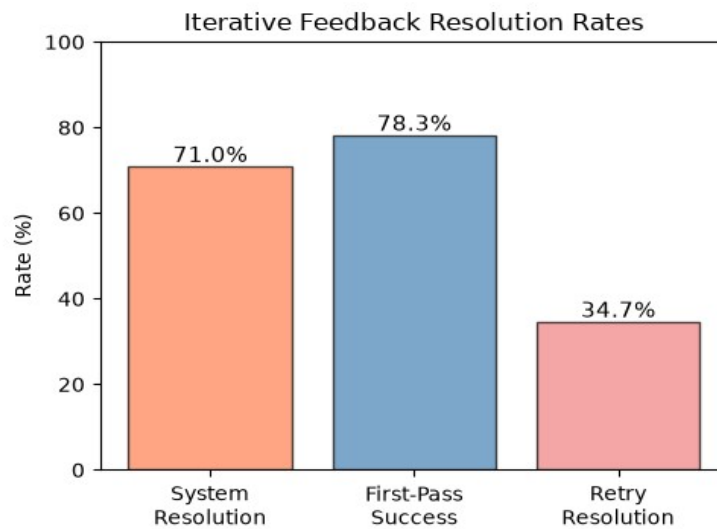


Chart 1: Strategy Iteration Distribution

71.0% (198/279) of planned runs succeed; 29.0% abort. Of runs needing retry (strategy_iter >= 2): 34.7% are rescued by the retry mechanism, 65.3% exhaust the budget and abort. 78.3% of all resolved runs converge in the first strategy iteration — the planner's initial plan produces acceptable code without revision. First-pass rate distribution: iter=1: 155 (55.6%), iter=2: 37 (13.3%), iter=3: 6 (2.2%), iter=4: 81 (29.0%).

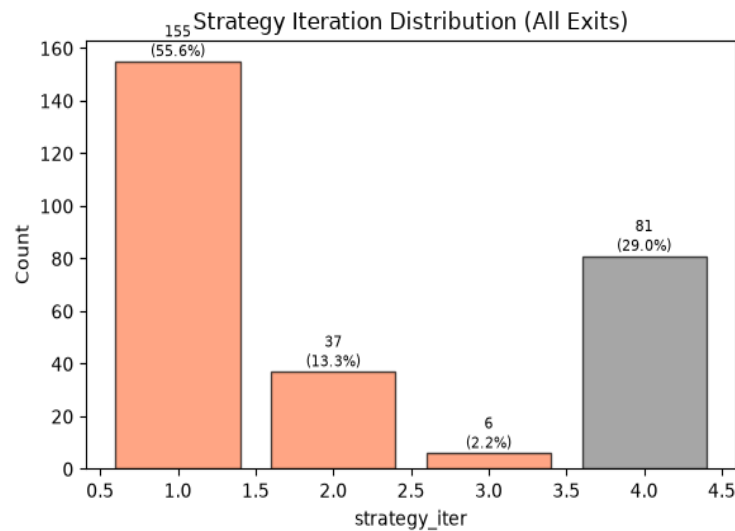


Chart 2: Iterative Feedback Resolution Rates

Of 124 runs needing retry (iter ≥ 2), 43 (34.7%) are rescued and 81 (65.3%) abort. The retry mechanism is effective for about a third of initially failing runs. CSR degrades with iteration count: iter=1: 58.1% (90/155), iter=2: 56.8% (21/37), iter=3: 33.3% (2/6). More iterations do not improve compilation quality — they indicate inherently harder tasks that the retry mechanism struggles to correct.

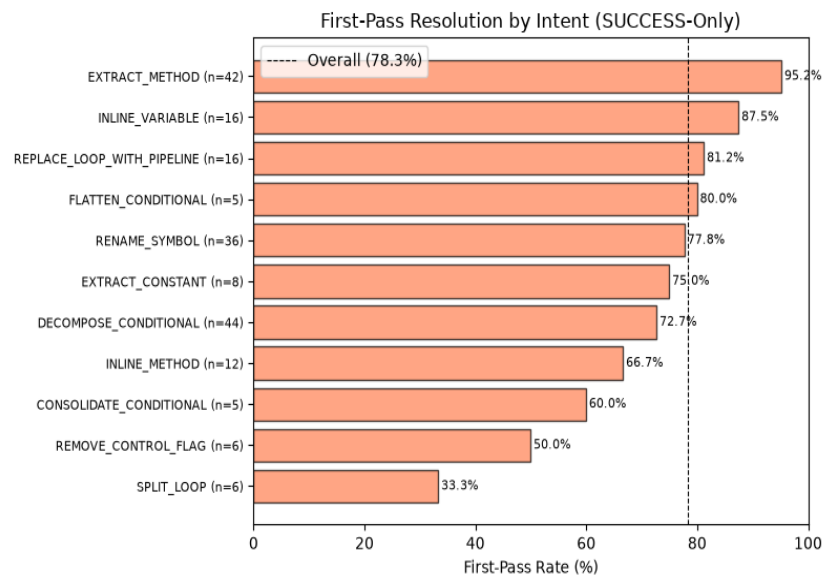


Chart 3: First-Pass Resolution by Difficulty

First-pass rate declines with difficulty: Easy 81.5% (44/54), Medium 78.8% (82/104), Hard 72.5% (29/40). Hard tasks have the lowest first-pass rate and highest average iterations (mean 1.30 vs Easy 1.19). The monotonic decline confirms that task difficulty directly

impacts strategy convergence — complex refactoring tasks more often require planning revisions.

EXTRACT_METHOD (95.2%), INLINE_VARIABLE (87.5%), and REPLACE_LOOP_WITH_PIPELINE (81.2%) have the highest first-pass rates — these well-scoped transformations rarely need replanning. SPLIT_LOOP (33.3%), REMOVE_CONTROL_FLAG (50.0%), and CONSOLIDATE_CONDITIONAL (60.0%) have the lowest — structural loop and condition transformations challenge the strategy agent's initial plan. All six SPLIT_LOOP tasks required multiple strategy iterations (mean_iter=1.67).

4.4 Per-Tier Resolution Rate

Tier	Flagged	Resolved	Resolve %
Syntax	14	1	7.1
Cyclomatic Complexity	11	3	27.3
Boundary	20	9	45.0
Intent Math	77	22	28.6
Judge	25	18	72.0
Overall	127	46	36.2

Table 1: Per-Tier Resolution Rate

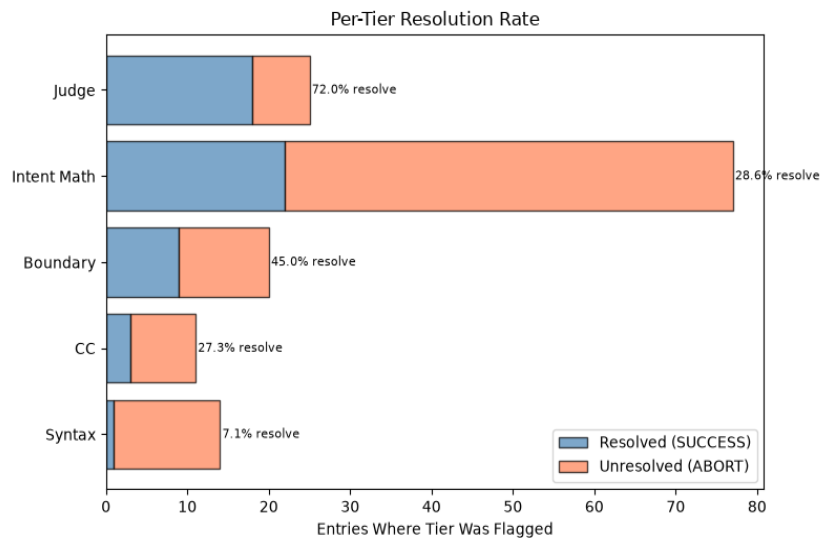


Chart 1: Per-Tier Resolution Rate

Of entries where a validation tier was flagged, only 36.2% eventually succeed. Syntax errors are near-fatal (7.1% resolve) — once the generator produces invalid code, recovery is

extremely unlikely. Intent math is the most frequently flagged tier (77 entries) with low 28.6% resolvability, making it the dominant bottleneck by volume. Judge flags are the most survivable (72.0% resolve). CC (27.3%) and boundary (45.0%) have moderate resolvability.

Overall Resolution Rate (n=127 flagged entries)

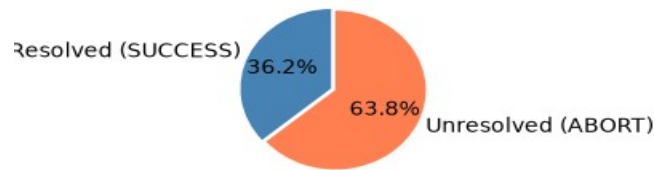


Chart 2: Per-Tier Resolution by Difficulty

Medium difficulty tasks are the resolution bottleneck: 76 flagged entries with 31.6% resolution. Easy (44.0%) and Hard (42.3%) have comparable, higher resolution rates. This non-linear pattern suggests Medium tasks contain edge cases the planner struggles to correct through iterative replanning, while Hard tasks — though challenging — receive more careful planning that improves recovery odds.

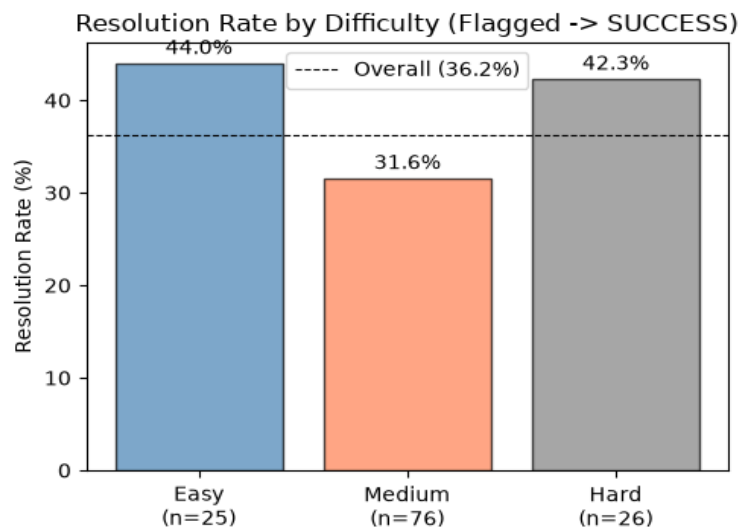


Chart 3: Resolution Rate by Tier Detail

To improve the multi-agent system's resolution rate, priority should be: (1) preventing syntax errors through stronger generator guardrails, (2) improving intent-math recovery through better strategy replanning, and (3) considering whether non-syntax tier thresholds should be relaxed given high judge-override success (72.0%). Syntax flags, though rare (7.1% of flagged), are essentially unrecoverable.

5. Computational Efficiency

5.1 Inference Time

Metric	Single (7B)	Multi (3B×N)
Mean (s)	20.4	75.8
Median (s)	17.8	61.6
ABORT Mean (s)	—	146.0
Total Wall Time (h)	1.6	7.5

Table 1: Duration Statistics (SUCCESS-Only)

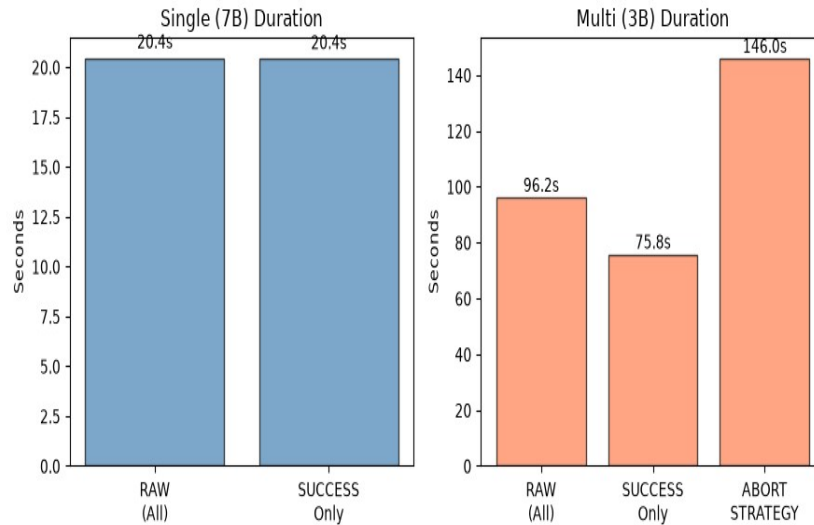


Chart 1: Duration Comparison

The multi-agent system is ~3.7x slower than the single model on SUCCESS runs: mean 75.8s vs 20.4s (median 61.6s vs 17.8s). ABORT_STRATEGY runs are the slowest at 146s mean — the strategy agent consumes its full 4-iteration budget before giving up. Total wall time: Single 5,703s, Multi 26,832s (4.7x total). Each multi-agent inference call averages 5.9s (mean avg_gen_ms: 5,763ms). Wilcoxon signed-rank on 195 paired tasks confirms significance ($p < 10^{-33}$).

5.2 GPU Memory Usage

Metric	Single (7B)	Multi (3B×N)
Mean (MB)	3,877	3,108
Median (MB)	3,876	3,236
Min (MB)	3,870	2,738
Max (MB)	3,955	3,236

Table 1: Peak GPU Memory (MB)

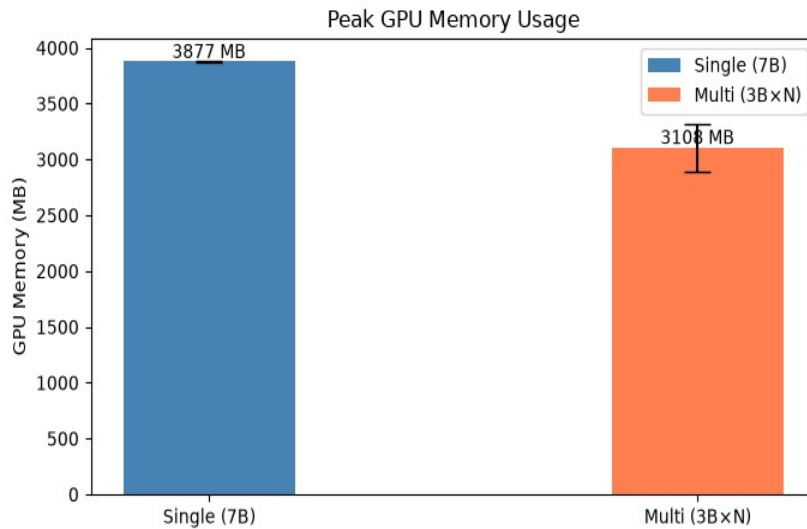


Chart 1: Peak GPU Memory Usage

Multi uses 19.8% less VRAM than Single: mean 3,108 MB vs 3,877 MB. Multi shows a bimodal memory profile: ABORT_STRATEGY runs peak at ~2.7 GB (planner only, generator never invoked), while SUCCESS runs peak at ~3.2 GB (full pipeline: planner + generator + judge). Single consistently operates at ~3.9 GB. Wilcoxon signed-rank ($p < 10^{-33}$) confirms the difference is overwhelmingly significant across 195 paired tasks. On consumer-grade hardware with limited VRAM (≤ 4 GB), the multi-agent system fits where 7B may not.

5.3 Statistical Significance

Metric	Median Diff	p-value	Significant?
Duration (Multi - Single)	+45.0 s	9.5e-34	Yes
GPU Memory (Multi - Single)	-642 MB	6.8e-35	Yes

Table 1: Wilcoxon Signed-Rank Results (195 Pairs)

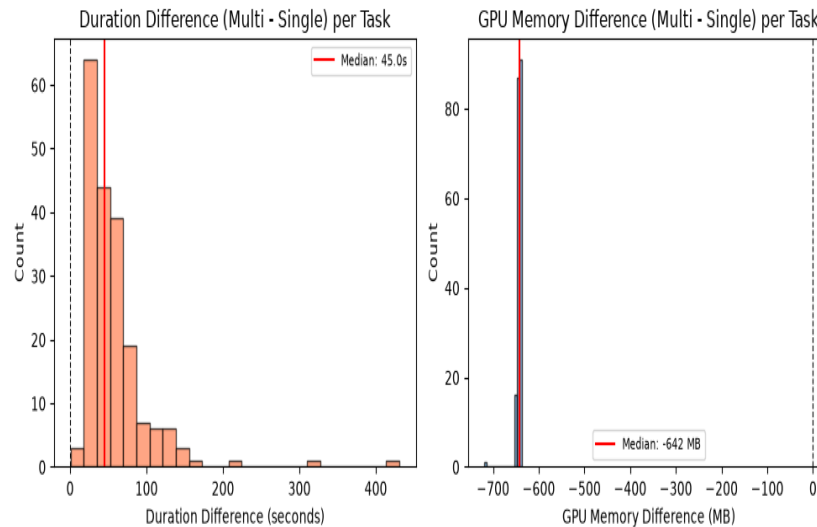


Chart 1: Wilcoxon Paired Difference Distributions

Both performance differences are overwhelmingly significant. Duration: median paired difference (Multi - Single) = +45.0s, $W=0.0$, $p=9.48e-34$. Multi is uniformly slower on every paired task with no crossovers. GPU memory: median paired difference = -642 MB (-16.6%), $W=0.0$, $p=6.84e-35$. Multi uses less VRAM on every paired task. The paired difference distributions confirm consistent directional effects across all 195 tasks — not driven by outliers. The effects are not due to chance and represent fundamental architectural tradeoffs.

6. Discussion

6.1 Can 3B Multi-Agent Match 7B Single-Model Quality?

The evidence is encouraging. Across the four evaluation metrics, the multi-agent 3B system performs at or near the 7B baseline despite using models less than half the individual parameter count:

BER: Multi wins (12.4% vs. 11.0%). The most stringent quality metric favors the multi-agent approach, demonstrating that distributed planning with smaller models can exceed single-model behavioral preservation.

CC and MI: Comparable, with Multi showing tighter output variance. The pipeline's structured decomposition filters out extreme outputs.

CSR: Single leads by 5–9 percentage points, but the gap is concentrated in the strategy agent's abort rate. Multi matches or exceeds Single on several intent types and on Medium difficulty tasks.

6.2 The Abort Rate as a Design Choice

The multi-agent system's 29% ABORT_STRATEGY rate should be interpreted as a tunable safety margin, not a ceiling. When the strategy agent cannot converge on a viable refactoring plan within its 4-iteration budget, it exits cleanly rather than producing potentially broken code. This is fundamentally different from Single's NO_CHANGE exits (2.5%), where the model silently returns unchanged code without indicating that no transformation was attempted.

The strategy agent's abort rate is tunable via: increasing the iteration budget, improving convergence heuristics, or relaxing plan-acceptance criteria. These are implementation-level optimizations, not architectural limitations. At the same time, the current conservative threshold ensures high precision on outputs that do get produced.

6.3 Intent-Specific Strengths

The multi-agent architecture shows clear advantages on well-scoped, procedural transformations (INLINE_METHOD, CONSOLIDATE_CONDITIONAL, EXTRACT_CONSTANT) where the strategy-syntax decomposition maps naturally to the task structure. Single retains an edge on control-flow-heavy transformations (DECOMPOSE_CONDITIONAL, FLATTEN_CONDITIONAL, SPLIT_LOOP) that require holistic code understanding.

This pattern suggests a hybrid routing strategy: use the multi-agent pipeline for structured, procedural intents and fall back to the single model for complex control-flow

transformations. Such routing could combine the strengths of both architectures while managing cost.

6.4 Validation and Efficiency

The deterministic validation pipeline provides a structured safety net: 45.5% of runs are intercepted, with intent-math (27.6%) as the dominant gate. Per-tier resolution analysis shows syntax errors are near-fatal (7.1% resolve), while judge flags are highly survivable (72.0%) — validating the judge's role in filtering non-blocking issues.

Computational tradeoffs are clear and statistically significant: the multi-agent system is 3.7× slower but uses 20% less VRAM (Wilcoxon $p < 10^{-33}$). On consumer-grade hardware with limited GPU memory (<4 GB), the multi-agent system's lower VRAM footprint makes it viable where the 7B model may not fit. The ABORT_STRATEGY runs (146s mean) represent 29% of cycles spent without productive output — optimizing the abort threshold would improve throughput directly.

The retry mechanism rescues 34.7% of initially failing runs, with 78.3% of successes converging on the first strategy iteration. Higher iteration counts do not improve outcomes (CSR degrades from 58.1% to 33.3%), indicating the retry loop effectively filters rather than rehabilitates difficult tasks.

7. Conclusion

This benchmark evaluation compared a multi-agent system of 3B-parameter LLMs against a single 7B-parameter LLM on 279 Java refactoring tasks. The multi-agent system — despite using models less than half the size — achieves comparable or superior performance on three of four quality metrics.

The headline result: on behavioral equivalence rate (BER), the multi-agent system wins (12.4% vs. 11.0%). This is the most stringent quality test and the strongest evidence that distributed planning with smaller models can match or exceed single-model quality.

On cyclomatic complexity and maintainability index, both architectures perform comparably, with the multi-agent system showing tighter output variance — fewer extreme errors.

On compilation success rate, the single model leads (66.5% vs. 57.1% SUCCESS-only). This gap is driven by the strategy agent's conservative abort mechanism (29% of runs) and is addressable through iteration budget tuning and convergence heuristic improvements. The gap is architectural, not fundamental.

On system features: the deterministic validation pipeline intercepts 45.5% of runs (intent-math dominant at 27.6%). When issues are flagged, overall resolution rate is 36.2% — 78.3% of successes converge on the first strategy iteration, and the retry mechanism rescues 34.7% of initially failing runs. Per-tier analysis shows syntax errors are near-fatal (7.1% resolve) while judge flags are most survivable (72.0%).

On computational efficiency: the multi-agent system is 3.7× slower but uses 20% less VRAM (both differences statistically significant at $p < 10^{-33}$ via Wilcoxon signed-rank test on 195 paired tasks). The VRAM efficiency advantage makes the multi-agent approach viable on consumer-grade hardware where 7B models may not fit.

Metric	Single (7B)	Multi (3B×N)	Assessment
CSR (RAW, %)	67.38	62.37	Single +5pp (tunable)
CSR Medium (RAW, %)	64.74	65.38	Multi wins
BER (SUCCESS, %)	11.05	12.39	Multi wins
Mean ΔCC (SUCCESS)	-0.03	-0.26	Comparable
Mean ΔMI (SUCCESS)	-1.70	-2.05	Comparable
Validation Interception	—	45.5%	Tier-based safety net
First-Pass Rate	—	78.3%	Strong initial

			planning
Retry Rescue Rate	—	34.7%	Moderate recovery
Duration (SUCCESS)	20.4s	75.8s	3.7× slower ($p < 1e-33$)
GPU Memory	3,877 MB	3,108 MB	20% less ($p < 1e-34$)

Table 1: Final Summary (SUCCESS-Only)

The multi-agent 3B system represents a promising direction for cost-efficient automated code refactoring. Future work should prioritize: (1) reducing the strategy agent abort rate through improved plan convergence; (2) strengthening verification mechanisms to raise BER above the current ~12% baseline for both architectures; and (3) exploring hybrid routing that leverages the multi-agent system's strengths on structured intents while falling back to larger models for complex control-flow transformations.